UNIFIED SYSTEM ARCHITECTURE - INTEGRATION BLUEPRINT

# One Machine from Seven Repos

Every repository you supplied, joined into one real-world system: the five-layer architecture, the contract between every pair of components, the end-to-end lead journey, integration sprints 5-8, nine verticals on one pipeline, and the guardrails that keep it alive.

## Standard-autonomous Stack - The Map of the Machine

Repo Inventory - Five Layers - Contract Matrix - Sprints 5-8 - Verticals - Guardrails

OCTOBER 2026

# Table of Contents

1. The Machine You Are Building 1

2. The Complete Repo Inventory - What You Gave Me and What Each Piece Does 2

3. The Five-Layer Target Architecture 5

4. One Lead's Journey - The Real-World Flow End to End 7

5. The Integration Contract Matrix - Who Talks to Whom, and How 8

6. Real-World Problems - Which Part Solves Them 10

7. The Integration Build Order - Wiring the Repos After the 4-Week Fix 11

8. Nine Verticals, One Architecture 13

9. Guardrails - What Can Kill This Machine and the Countermeasure for Each 15

10. The Master Library - Every Deliverable and When to Open Which 16

# 1. The Machine You Are Building

You did not hand me seven unrelated repositories; you handed me the parts of one machine. On their own, each piece is a tool: a scraper here, a CRM there, an inbox somewhere else. Connected by typed contracts, they become a single revenue system that finds real businesses, learns what is broken in their online presence, proves it with evidence, reaches the owner personally on WhatsApp in their own language, and remembers every touch from first discovery to closed deal. This document is the blueprint of that machine: the complete inventory of what you gave me, the five-layer architecture that connects it, the exact integration contract between every pair of components, and the sprint-by-sprint build order that wires it together after the four-week repair of the IC core.

The one-sentence architecture thesis is this: **IC is the brain, the scrapers are the hands, Twenty is the memory, Chatwoot is the ears and mouth, and hard CI rules with the trust curve are the conscience.** Every design decision in the chapters that follow is a consequence of that sentence. When you wonder where a new capability belongs, you do not ask "where does the code go"; you ask "which organ does this job". A new data source is another hand. A new report is another memory write. A new message template is another sentence the mouth can speak. A new automation without a proof requirement is a reflex the conscience must block.

The machine exists to solve seven concrete problems that every lead-generation business hits, and today solves each of them with manual labor. First, **where do leads come from** - answered by three-tier discovery that turns a city grid into a queue of verified businesses. Second, **what is actually wrong with their online presence** - answered by the REA audit running a real iPhone-14 viewport with screenshots. Third, **how do I prove it instead of guessing** - answered by the pstack evidence engine, where every claim carries a proof score. Fourth, **how do I reach a shop owner in Mumbai at 11 a.m. without sounding like a robot** - answered by the Hinglish copywriter and the WhatsApp send path. Fifth, **how do I never lose track of a conversation** - answered by Twenty CRM with a full timeline per lead. Sixth, **how do I scale past one vertical** - answered by per-vertical configuration and eval sets instead of new code. Seventh, **how do I trust an autonomous system with my market reputation** - answered by governance: hard CI rules, the nightly audit engineer, and a five-stage trust curve that earns automation with measured evidence.

| 7 | 5 | 20+ | 9 |
|---|---|---|---|
| repos and stacks integrated into one system | architectural layers, each replaceable | typed integration edges, all with failure modes | verticals served by the same pipeline |

One principle separates this design from the integration diagrams that die in slide decks: **layers, not a monolith.** Each layer has exactly one job, a typed output, and a declared failure mode. If gosom gets blocked by Google tomorrow, Layer 1 keeps working on Places and Overpass. If Chatwoot is down,

sends queue in the outbox instead of vanishing. If you outgrow Twenty, the mirror pattern means the migration touches one layer. The audit of your repository showed the brain is already industry-grade - contracts, evidence chains, and the ECC shield are Cursor-level design. What was missing is the plumbing that connects the organs, and that plumbing, not any new feature, is what the remaining chapters specify.

# 2. The Complete Repo Inventory - What You Gave Me and What Each Piece Does

Before wiring anything, every part must be named, its role fixed, and its honest current status recorded - because integration planning built on wishful thinking produces week-three surprises. The inventory below groups everything you gave me into four roles. **Core system** is the codebase that becomes the brain. **Discovery and enrichment stack** is the set of hands. **Business systems** are the organs you should deploy rather than rebuild. **Methodology donors** shape how the team works without shipping in the runtime. For each entry the table states what it is, what job it does in the machine, and its integration status inside IC today, verified during the audit.

## 2.1 Core system: Standard-autonomous (IC, the Intelligence Center)

| Module group | Location | What it contributes | Status today |
|---|---|---|---|
| Contracts and schemas | src/core/contracts.py, src/models/ | Typed LeadRecord, HandoffEnvelope, AuditReport - the vocabulary every layer speaks | **Healthy** - the strongest asset |
| Evidence engine (pstack) | src/services/evidence_verifier.py | VerifiedClaim + proof score; claims require proof | Healthy, unenforced without CI |
| ECC shield | src/agents/ecc_shield.py | Outbox gate that blocks unproven or unsafe sends | Healthy; gate wired to approval |
| Pipelines | src/pipelines/ (discovery, chat, outreach) | Eight-stage flow from discovery to CRM loop | Depends on missing storage layer |
| Scrapers (v2 wrappers) | src/scrapers/ (base, gmaps, firecrawl, scrapling, gosom, web_discovery) | Adapter pattern over every data source | Wired but **degraded**: deps missing |
| Cadence bots | src/services/scheduled_cadence.py | ProjectsManagerBot, NightlyAuditEngineer, QABot on a schedule | Healthy; modeled on Lauren Tan's marketplace |

| Module group | Location | What it contributes | Status today |
|---|---|---|---|
| Inbound closer | src/services/inbound_closer.py, workforce.py | Reply classification and ECC-checked drafts | Healthy; needs Chatwoot events to trigger |
| Storage layer | src/storage/repository.py | LeadRepository and SessionRepository | **Missing from git** - Week 1 rebuild, 5 test files cannot import |
| API + dashboard | src/api/server.py, templates/ | FastAPI, four-step UI, token system | Works; zero auth until Week 3 |

Table 2.1 - The IC core: what each module group contributes to the machine and its verified status.

Two facts from the audit matter for every integration decision that follows. First, the five test files that fail to import all fail for one reason: the storage module was never committed, so the Week 1 rebuild of LeadRepository is the prerequisite for trusting any pipeline end to end. Second, the scraper wrappers exist and are sensibly designed, but scrapling and firecrawl are absent from requirements.txt, so out of the box the system silently runs fallback paths instead of the real tools. Both facts mean the machine's organs are present; the connective tissue and the truth about dependencies are what the build order restores.

## 2.2 Discovery and enrichment stack: the hands

| Component | What it is | Job in the machine | Integration status |
|---|---|---|---|
| Google Places API | Paid, ToS-compliant business data service | Tier 1 discovery default: structured fields, stable place_id for dedupe | Primary path in src/scrapers/gmaps.py |
| OSM Overpass | Free OpenStreetMap query API | Tier 2 fallback for coverage gaps and zero-cost testing | Wired as secondary source |
| gosom google-maps-scraper | Docker sidecar driving headless browsers against Google Maps | Tier 3 bulk: whole-city grid crawls and email harvesting Places does poorly | Wrapper exists; ToS-cautioned, low concurrency, off-peak only |
| Scrapling | BSD-3 Python library; TLS-impersonating fetcher, stealth fetch, adaptive selectors | Default fetch/parse: contact pages behind Cloudflare, markdown for the LLM, self-healing selectors | Wrapper exists; **absent from requirements.txt** |
| Firecrawl | AGPL core + cloud API; site mapping, actions, structured extraction | /v2/map to enumerate a prospect site, batch scrape with actions, extract into AuditReport fields, change monitors | Client exists; same missing-deps problem |
| ScrapeGraphAI | LLM-orchestrated scraping graphs | Special-cases only: one-off odd sources, never the firehose (slow, costly) | Recommended, not wired |

Table 2.2 - The discovery and enrichment stack: three tiers of sources plus the two enrichment workhorses.

## 2.3 Business systems: deploy, do not rebuild

| System | What it is | Job in the machine | Integration status |
|---|---|---|---|
| Twenty | Open-source CRM (NestJS + GraphQL + Postgres, React front) built with SDR work in mind | System of record: Company, Person, Opportunity stages New-Audited-Contacted-Replied-Negotiating-Won, timeline, notes | New integration - Sprint 6 |
| Chatwoot | Open-source conversation platform (Rails + Postgres/pgvector + Redis, one docker-compose) | Two-way WhatsApp inbox: per-client accounts, campaign inboxes, message_created webhooks into the Closer | New integration - Sprint 7 |
| Twilio WhatsApp | Messaging API for WhatsApp Business templates | The compliant send path: approved templates, quiet hours, per-tenant ceilings | Existing outbound module + gateway pattern |
| treg (pattern) | Tool-gateway pattern: task-based tool search, server-side secrets, price metadata, idempotency keys | One internal service holds every vendor key so no agent ever holds a credential | Pattern adopted - Sprint 5 |

Table 2.3 - Business systems: buy-and-deploy organs that IC integrates through typed edges.

## 2.4 Methodology donors: how the team works, not what ships

| Donor | What it gives the team | Where it shows up |
|---|---|---|
| Lauren Tan masterclass (Cursor) | Verification-first loop: trust agents with verification, then hard CI rules, then auto-merge; single-mandate bots | scheduled_cadence.py is explicitly modeled on it; Master Action Plan Ch. 7 trust curve |
| harness2 (Google Research) | Propose-probe-compose monthly loop on agent prompts; keep only measured wins | Monthly Closer prompt review against the golden eval set |
| pstack discipline | Claims require proof - no number ships without an artifact | evidence_verifier.py; every pitch line traces to a ProofBundle |
| 51 vendored skills (.agents/skills/) | superpowers, ECC, mattpocock-skills, treg skill, wayfinder and more | Team standards referenced in code docstrings |

Table 2.4 - Methodology donors: they set how the machine is run and improved, not runtime code.

The honest summary of this inventory in one sentence: **you already own every organ of the machine; what you do not yet own is the connective tissue.** The scrapers are wired but undeclared, the inbox and

CRM are named but not deployed, the gateway is a pattern but not a service, and the storage layer is specified by five failing tests but not committed. Nothing in the chapters that follow asks you to buy or build a new core component. Every sprint either declares an existing dependency, deploys a named open-source system, or writes the typed edge between two organs that already exist. That is why the architecture is realistic: it is assembled almost entirely from parts already on your disk.

# 3. The Five-Layer Target Architecture

The figure below is the whole machine on one page. Four working layers flow left to right - Discovery feeds Intelligence, Intelligence writes Memory, Memory grounds Engagement - and a fifth governance layer spans everything underneath, observing rather than participating. Solid arrows are the lead's forward path; the dashed green return path is how replies flow back from Engagement into Intelligence so that a broadcast becomes a conversation; dashed taps downward are how every layer reports to governance. Each layer's output is named inside the figure, and those outputs are typed contracts, not informal handoffs - the same discipline your contracts.py already applies inside IC, extended across the whole system.

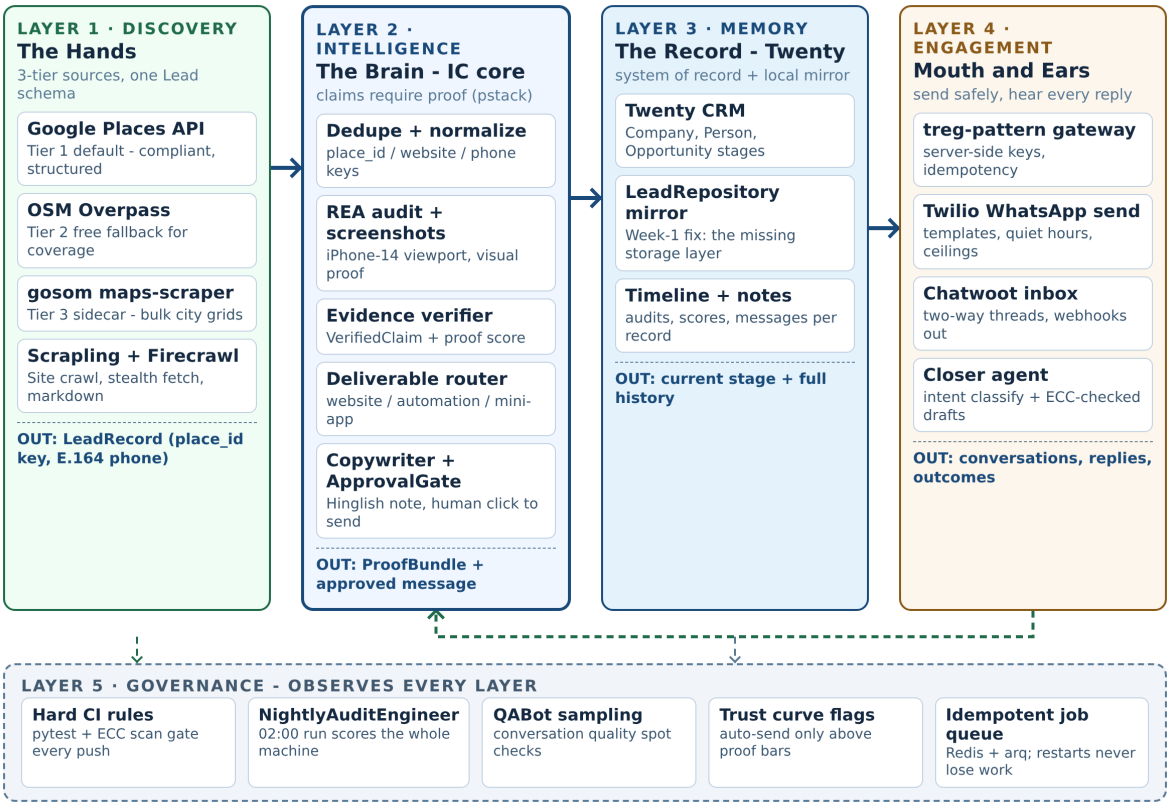



Figure 1: The five-layer integration architecture. Solid arrows: lead flow. Dashed green: reply feedback. Dashed taps: governance observation.

## 3.1 Why layers, and what each one guarantees

**Layer 1 - Discovery (the hands)** exists to turn "the market" into a typed queue. Its three-tier design is a deliberate insurance policy: Google Places is the compliant default, OSM Overpass covers its gaps for free, and the gosom sidecar does bulk grid crawls that Places prices poorly - run at low concurrency, off-peak, with proxies, because it drives a browser against Google Maps and that is ToS-cautioned territory. Scrapling and Firecrawl then fetch what Places cannot: contact pages behind Cloudflare, full site maps, markdown for the LLM. The layer's single output is a LeadRecord keyed on place_id with an E.164 phone, which is what makes every downstream layer reliable - dedupe is only possible when the key is stable.

**Layer 2 - Intelligence (the brain, IC core)** is where a raw business becomes a proven opportunity. Deduplication and normalization run first; the REA audit loads the site in a real iPhone-14 viewport and captures screenshots as visual evidence; the evidence verifier turns observations into VerifiedClaim objects with proof scores; the deliverable router decides whether the dominant pain points to a website, an automation, or a mini-app; and the copywriter drafts the Hinglish growth note grounded only in verified claims. Nothing leaves this layer without passing the ApprovalGate - a human click. The layer's output is a ProofBundle plus an approved message, and that pair is the reason the machine can be trusted: the pitch and the evidence travel together.

**Layer 3 - Memory (the record, Twenty)** answers "what happened with this business?" forever. Twenty is deployed, not rebuilt, because its data model already anticipates sales development: Company and Person records, Opportunity stages, a timeline object that accepts every event you push. The LeadRepository mirror rebuilt in Week 1 sits in front of Twenty as the local write-through cache, so IC reads are fast and a Twenty outage degrades to queuing rather than data loss. **Layer 4 - Engagement (mouth and ears)** sends through the treg-pattern gateway so vendor keys never live in agent code, delivers via Twilio WhatsApp templates inside quiet hours and ceilings, and receives through Chatwoot, whose webhooks wake the Closer agent for every reply. **Layer 5 - Governance** owns the conscience: CI gates every push, the NightlyAuditEngineer scores the whole machine at 02:00, QABot samples conversations, trust-curve flags decide what may auto-send, and the idempotent job queue guarantees restarts never lose work.

> *The spine of the whole architecture is one sentence: claims require proof. A lead without a source is not a lead; an audit without a screenshot is not an audit; a message without a ProofBundle does not leave the outbox.*

The layers earn their keep in failure, not in the happy path. When Google blocks the sidecar, Layer 1 still ships Places and Overpass leads. When Firecrawl credits run out mid-week, Scrapling's markdown fetch covers enrichment until the quota resets. When Chatwoot restarts, sends queue behind the outbox gate and thread on recovery. When a new teammate joins, they can be given Layer 1 alone as a well-defined project without reading a line of the Closer. That replaceability - each layer a named organ with one job

and typed inputs and outputs - is what turns seven repositories into one system that survives contact with the real world.

# 4. One Lead's Journey - The Real-World Flow End to End

Architecture only matters when you can trace one real business through it. The walkthrough below follows a plausible lead end to end - a Mumbai dental clinic, the same vertical your repository already seeds in data/harness_library and data/screenshots. Every numbered step is an edge from Figure 1, so as you read, you are also verifying the integration matrix of Chapter 5. The point to notice: at no step does data travel informally. Each handoff is a typed payload with a defined failure behavior, which is exactly what makes the machine debuggable when something goes wrong at 2 a.m.

1. **Discover (L1).** The morning discovery job queues a Bandra-West grid crawl. Google Places returns the clinic with place_id, rating, phone in E.164, and website; the job writes a LeadRecord to the queue. If Places errors, the Overpass fallback runs; the source is recorded on the lead either way.

2. **Dedupe and normalize (L2).** The enrichment worker checks place_id against Layer 3 - already known, so this becomes an update, not a new lead. Phone normalizes to E.164; the website host is parsed for the audit.

3. **Fetch and crawl (L1 to L2).** Scrapling fetches the contact page with a real browser TLS fingerprint; Firecrawl maps the site and pulls the booking page as markdown. The LLM now knows the clinic has no online booking, a phone-only contact flow, and a menu written for 2015.

4. **REA audit (L2).** The site inspector loads the site in an iPhone-14 viewport: 6.8-second load, layout shift pushing the phone number below the fold, no SSL on the booking form. Three screenshots are saved to data/screenshots/ as proof artifacts.

5. **Evidence verification (L2).** Each observation becomes a VerifiedClaim: "6.8s mobile load (measured 3 runs)" PROVEN_TRUE with score 0.95; "no SSL on booking" PROVEN_TRUE; "loses mobile bookings" INFERENCE_WITH_SUPPORT. The ProofBundle hashes the screenshots and records the source URLs.

6. **Route and draft (L2).** The deliverable router sees a broken mobile booking flow and picks website-build-with-booking-automation. The copywriter writes the Hinglish growth note - three sentences, every number traceable to the bundle, opt-out line included - and files it in the outbox.

7. **Human approval (L2 gate).** The outbox shows the note side-by-side with the WhatsApp preview and the proof list. You read the three claims, click approve. The ECC gate stamps the bundle id and the message enters the send queue.

8. **Remember (L3).** The approval event lands on the Twenty timeline: Opportunity New to Contacted, note with bundle reference attached. Nothing is remembered twice - the idempotency key is the bundle id.

9. **Send (L4).** The gateway injects the Twilio key server-side, checks quiet hours and the per-tenant ceiling, and sends the approved template to the clinic's WhatsApp at 11:07 IST. Retry on transport error is safe because the send is keyed.

10. **Hear and close the loop (L4 to L2 to L3).** The owner replies "kya cost hoga?" forty minutes later. Chatwoot fires message_created; the Closer classifies it as pricing-interest, drafts an ECC-checked reply from the ProofBundle, and queues it for your approval. The reply threads in the same Chatwoot conversation; the Opportunity moves to Replied in Twenty; the nightly audit scores the whole run for the morning review.

Two properties of this journey deserve emphasis. First, **the human is inside the loop at exactly two points** - approving the first message and approving the pricing reply - and everywhere else the machine is allowed to work unattended, because those are the two moments where a wrong word costs money or reputation. As the trust curve advances (Chapter 7 of the Master Action Plan), the second gate can earn auto-send for low-risk templates, but it is earned with measured evidence, never assumed. Second, **every step records its inputs**: when a week later the owner asks "what did you actually measure?", the answer is one bundle id away, not a memory exercise. That audit trail is what separates a sales machine from a spam cannon.

# 5. The Integration Contract Matrix - Who Talks to Whom, and How

This chapter is the engineering heart of the blueprint: every edge in Figure 1 written down as a contract. For each integration the table names the producer, the consumer, the mechanism, the payload that must survive the wire, and the declared failure mode - because the difference between a system and a demo is that the system's author knows exactly what happens when each link breaks. Three mechanisms cover all twenty-odd edges: typed JSON contracts for in-process and queue traffic, REST/GraphQL with idempotency keys for services, and webhooks for event push. If a proposed feature does not fit one of the three, it is a design smell worth resisting.

| From | To | Mechanism | Payload contract | Failure mode |
|---|---|---|---|---|
| Scraper adapters | IC queue | Typed JSON on job queue | LeadRecord: place_id, name, phone E.164, website, source, fetched_at | Source error: skip, log, count in nightly score |
| IC dedupe | Layer 3 | Idempotent upsert | place_id primary key; website host, phone as secondary keys | Conflict: keep earliest, mark for review |
| IC audit | Evidence store | Typed contract | VerifiedClaim list + proof score + screenshot hashes + source URLs | Unproven claim: rejected, never reaches copy |

| From | To | Mechanism | Payload contract | Failure mode |
|---|---|---|---|---|
| IC router + copywriter | Outbox | Typed contract | DeliverableChoice + draft message + ProofBundle reference | ECC gate rejection: draft returned with reason |
| ApprovalGate | Send queue | Outbox pattern | Approved message + bundle id as idempotency key | Never auto-sends below trust-curve stage bar |
| IC | Twenty | REST + GraphQL, keyed writes | Company, Person, Opportunity stage events, timeline notes | Twenty down: queue writes; retry keyed, no duplicates |
| Twenty | IC | Webhook | Stage-change events (opportunity.updated) | Becomes a follow-up task in the job queue |
| Chatwoot | IC Closer | Webhook | message_created, conversation_status_changed | Missed webhook: reconciliation sweep each hour |
| IC | Chatwoot | API-channel inbox | Outbound message mirror with thread id | Send fails: outbox retry; thread id preserved |
| IC agents | treg gateway | Internal tool call | Task + params + idempotency key; no credentials in payload | Gateway injects keys; retry never double-bills |
| Gateway | Twilio | WhatsApp template send | Approved template + variables + tenant ceiling check | Quiet hours: hold; ceiling: defer to next window |
| Cadence bots | IC services | Scheduled runs (02:00) | Nightly audit report; QABot samples; project board updates | Run failure: flagged in morning review, not silent |

Table 5.1 - The integration contract matrix: every edge, its payload, and its declared failure mode.

**The dedupe key strategy deserves its own paragraph because it quietly decides data quality for the whole machine.** The primary key is Google's place_id: stable, issued by the source, and present on every Tier 1 lead. When a lead arrives without one - Overpass results, sidecar crawls - the secondary composite key takes over: normalized website host, then E.164 phone. Normalization rules are unglamorous but decisive: strip spaces and punctuation from phones and prefix the country code, lowercase and strip "www." from hosts, and treat missing website as "phone-only lead" rather than inventing a placeholder. On conflict - two sources, slightly different names - the earliest verified record wins and the challenger is marked for review rather than merged silently, because a silent merge is how duplicate pipelines and wrong timelines are born. These rules are fifteen lines of code and a week of arguments saved per quarter.

One more contract rule that costs nothing now and saves a migration later: **every payload carries its schema version.** When Layer 2 adds a field to LeadRecord, Layer 3 jobs written last month must not crash at 02:00. The convention is a two-line addition - "schema": 1 on write, tolerant read on consume - and it converts every future change from a coordinated deploy into a backward-compatible rollout. The audit found your HandoffEnvelope already does this inside IC; the matrix simply extends the habit to every edge that leaves the process boundary.

# 6. Real-World Problems - Which Part Solves Them

An architecture is only real when a business owner can point at a daily frustration and get a component name back. This chapter is that translation table. Each row is a question you have actually asked in your own words during this project - where do leads come from, are they real, what do I say, how do I not get banned - mapped to the layer that answers it, the mechanism that does the work, and the metric that proves the answer is working. When something feels broken in daily operation, find the row here first; the row tells you which layer to open and which number to check before you touch any code.

| Real-world problem | Answering component | Mechanism | Metric that proves it |
|---|---|---|---|
| Where do leads come from? | Layer 1 (Discovery) | Three-tier sources feeding one LeadRecord queue | Verified leads per week per city grid |
| Are these businesses real? | Layer 2 + Layer 3 | Dedupe on place_id; evidence verifier stamps every lead | % leads carrying a ProofBundle |
| What do I say to them? | Layer 2 (REA + copywriter) | Quantified pain list from audit; Hinglish note grounded in proofs | % pitch claims PROVEN_TRUE |
| How do I reach them? | Layer 4 (gateway + Twilio) | WhatsApp templates, quiet hours, business-hours sends | Delivery rate per template |
| How do I not get banned? | Layer 4 + Layer 5 | Approved templates, opt-out line, per-tenant ceilings, warm-up | Zero template rejections; stable sender |
| How do I not lose track? | Layer 3 (Twenty + mirror) | Every event on the timeline; stage moves are keyed writes | 100% of touches queryable per lead |
| Who answers when they reply? | Layer 4 (Chatwoot + Closer) | message_created webhook; Closer drafts ECC-checked reply | First-response time under 5 minutes |
| How do I know it works? | Layer 5 (nightly audit + analytics) | NightlyAuditEngineer scores every stage; weekly scorecards | Reply rate; stage conversion per vertical |

| Real-world problem | Answering component | Mechanism | Metric that proves it |
|---|---|---|---|
| How do I scale to 9 verticals? | Layer 2 config + evals | Per-vertical config files and harness eval sets, not new code | New vertical live in under a week |
| How do I trust the automation? | Layer 5 (trust curve + CI) | Automation stages earned with measured bars; CI gates merges | Stage promotions with zero rollback events |

Table 6.1 - Problem to component mapping: every daily question has one owner layer and one proof metric.

Notice what the table does **not** contain: any row that requires new invention. Every mechanism named is either already in your repository (dedupe, evidence, ECC, cadence bots), a deployment of an existing open-source system (Twenty, Chatwoot), or a small service with a precise spec (the gateway). That is the practical meaning of "you already own every organ" from Chapter 2. The build order in the next chapter sequences exactly these mechanisms, and the metrics column doubles as the acceptance test for each sprint - when the number in the last column moves, the wiring behind it works.

# 7. The Integration Build Order - Wiring the Repos After the 4-Week Fix

This chapter assumes the Master Action Plan has done its job: Week 1 restored the storage layer and turned the test suite green, Week 2 installed hard CI rules, Week 3 closed the security holes, and Week 4 merged the three generations of the codebase into one house. You do not wire integrations onto a broken core - every sprint below would otherwise be built on the five failing test files and the silent fallback paths the audit documented. With the core solid, four sprints wire the organs together in dependency order: hands first, then memory, then mouth and ears, then scale. Each sprint names its goal, its tasks, its acceptance check, and its rollback, because a sprint without a rollback plan is a bet, not a plan.

## 7.1 Sprint 5 - Wire the hands (declare and verify the scraper stack)

1. Add scrapling and firecrawl to requirements.txt (or extras) and pin versions; remove the silent fallback so a missing dependency is a loud startup error.
2. Write smoke tests for each scraper adapter using recorded fixtures (one saved Places payload, one saved Firecrawl map result, one saved Scrapling fetch) so tests never hit the live web and never flake.
3. Build the treg-pattern gateway skeleton: one FastAPI service holding all vendor keys, tool registration with price metadata, idempotency keys on every call.
4. Route Twilio and Firecrawl calls through the gateway; agents now hold zero credentials.

**Acceptance:** scraper smoke tests pass in CI on every push; the gateway answers a health-check with the tool catalog; zero vendor keys appear anywhere under src/ or .env files outside the gateway's own config. **Rollback:** the adapters keep their existing fallback paths behind a feature flag for one sprint, so declaring

the dependency can be reverted by unpinning, not by rewriting.

## 7.2 Sprint 6 - Wire the memory (Twenty and the mirror)

1. Deploy Twenty with its docker-compose (Postgres + GraphQL + React front) on your VPS behind the reverse proxy.
2. Write the IC-to-Twenty sync worker: Company and Person upserts keyed on place_id, Opportunity created at first verified audit, stage events written through the LeadRepository mirror.
3. Backfill: import the leads already in data/leads/ so Twenty starts with real history, not an empty board.
4. Add the schema-version field to every outgoing payload (Chapter 5 convention).

**Acceptance:** a lead discovered end-to-end appears in Twenty within sixty seconds of its first verified audit, with the audit screenshots and proof scores on its timeline; killing Twenty for ten minutes mid-crawl loses nothing - writes queue and flush on recovery. **Rollback:** the mirror is write-through by design; if Twenty must be abandoned, the LeadRepository remains the system of local record and another CRM can be adapted behind the same worker.

## 7.3 Sprint 7 - Wire the mouth and ears (Chatwoot and the reply loop)

1. Deploy Chatwoot (docker-compose) and provision a Platform-API account per client; create one API-channel inbox per campaign.
2. Mirror every approved outbound IC message into Chatwoot so threads and assignment live in one place.
3. Subscribe to message_created and conversation_status_changed webhooks; each event becomes a Closer task in the job queue.
4. Write the Hinglish template pack: ten approved templates per vertical with opt-out lines, registered with Twilio, each referencing the fields a ProofBundle can supply.
5. Re-wire the ApprovalGate to the send path: approve in the outbox, gateway sends, Chatwoot mirrors, timeline updates - one flow, one idempotency key.

**Acceptance:** an inbound "kya cost hoga?" produces a Closer draft in the outbox within five minutes, ECC-checked and grounded in the lead's ProofBundle; your approved reply threads correctly in Chatwoot and moves the Twenty stage. Quiet-hours and ceiling checks are provable by forcing a violation in staging. **Rollback:** webhooks can be paused per inbox; outbound falls back to the pre-Chatwoot one-way sender for a day without losing templates or ceilings.

## 7.4 Sprint 8 - Nine verticals (scale by configuration)

1. Extract per-vertical config: search grids, pain keywords, pitch angles, deliverable mappings - one file per vertical, validated by a schema test.
2. Create a harness eval set per vertical (minimum twenty golden cases from real audits) and wire it into CI as the vertical's gate.
3. Add the weekly vertical scorecard to the nightly audit: discovery volume, proof coverage, reply rate, stage conversion.
4. Run the trust-curve review per vertical: each vertical earns auto-send stages independently, by its own measured evidence.

**Acceptance:** a ninth vertical added as config plus evals passes CI without a single line changed in src/ outside config/; its scorecard appears in the morning review. **Rollback:** a misbehaving vertical is disabled by removing its config file - the machine keeps running the other eight. The table below compresses the four sprints into one view you can pin next to the Master Checklist.

| Sprint | Goal | Main tasks | Acceptance check | Rollback |
|---|---|---|---|---|
| 5 | Hands | Declare deps; fixture smoke tests; gateway skeleton; route Twilio+Firecrawl | Scraper tests green in CI; zero keys outside gateway | Feature flag restores fallbacks |
| 6 | Memory | Deploy Twenty; sync worker; mirror; backfill data/leads | Lead visible in Twenty in 60s; outage loses nothing | Mirror remains local record |
| 7 | Mouth + ears | Deploy Chatwoot; mirror sends; webhooks to Closer; template pack; gate wiring | Reply to draft in 5 min; quiet hours provable | Pause webhooks; one-way fallback |
| 8 | Scale | Vertical configs; eval sets; scorecards; per-vertical trust review | New vertical = config + evals only; scorecard live | Remove config to disable |

Table 7.1 - The four integration sprints in one view: goal, tasks, acceptance, rollback.

# 8. Nine Verticals, One Architecture

The machine does not change shape between industries; only its configuration does. Your repository already carries the seeds: data/harness_library/ contains dental_healthcare, logistics_transport, event_banquets, and general playbooks, and the audit identified nine launch verticals with quantified pain. This chapter fixes the mapping once so Sprint 8 is transcription, not invention. The rule that keeps nine verticals from becoming nine codebases is stated first: **a new vertical is a config file plus an eval set, never a fork of the pipeline.** The moment vertical logic leaks into src/, CI can no longer promise that a dental change cannot break weddings - and that promise is the whole value of the layered architecture.

| Vertical | Dominant pain found | Pitch angle (Hinglish frame) | Deliverable route |
|---|---|---|---|
| Dental and healthcare | No online booking; slow mobile load; no reviews strategy | "patients phone 3 clinics - the one that answers fastest online wins" | Website + booking automation |
| Logistics and transport | Quotation by phone only; no tracking page | "har load ka quote WhatsApp par, tracking link ke saath" | Quote automation + mini-app |
| Event and banquets | No availability calendar; inquiry form dies in inbox | "dekho dates, book karo - bina phone tag ke" | Website + calendar automation |
| Wedding vendors | Instagram-only presence; no lead capture | "Instagram se booking tak - ek link jo sab kuch kare" | Landing page + capture flow |
| Salons and spas | Manual appointment diary; no-shows | "reminder bhejo, no-show aadha karo" | Booking + reminder automation |
| Gyms and fitness | No trial funnel; paper membership | "7-din trial link - naye members apne aap aayenge" | Mini-app trial funnel |
| Real estate brokers | Listings on WhatsApp status only | "listings ek page par - buyer khud filter kare" | Website + listings CMS |
| Auto dealers and repair | No service booking; inventory invisible | "service slot online, gaadi ka record digital" | Website + booking automation |
| Education and coaching | Admissions by foot; no inquiry follow-up | "har inquiry ka reply automation se - admission tak" | CRM setup + follow-up automation |

Table 8.1 - The nine launch verticals: pain, angle, and deliverable route per industry.

Each vertical also declares its **source priority and eval set**. Dental and salons are Places-rich, so Tier 1 carries them almost alone. Logistics and real estate hide on directory sites, so Scrapling and Firecrawl do more work there. Coaching institutes cluster in local listing sites where the gosom sidecar earns its keep. The eval set per vertical - twenty golden cases minimum, drawn from real audits - is what makes the trust curve meaningful per industry: a wedding pitch that earns auto-send says nothing about a logistics pitch, and the scorecards keep the evidence separate. That is why Sprint 8 adds scorecards before it adds automation stages: in this machine, scale is purchased with per-vertical evidence, exactly as the Lauren Tan loop prescribes.

# 9. Guardrails - What Can Kill This Machine and the Countermeasure for Each

This machine sends messages to real people through platforms that can ban you, using data scraped from sources with terms, on the authority of models that can hallucinate. The guardrails are not bureaucracy; they are the difference between a business asset and a liability that gets your number blocked in week six. Each risk below is rated honestly, paired with the countermeasure already designed into the architecture, and assigned to the layer that owns it. Two risks deserve special respect because they kill most outbound operations in their first quarter: WhatsApp template compliance and Google ToS exposure.

| Risk | Severity | Countermeasure | Owning layer |
|---|---|---|---|
| WhatsApp template rejection and sender bans | **High** | Only pre-approved templates; opt-out line in every first message; quiet hours per region; per-tenant daily ceiling; number warm-up ramp; quality-rating monitoring in the nightly audit | Layer 4 + Layer 5 |
| Google Maps ToS exposure (gosom) | **High** | Places-first policy; sidecar runs tier-3 bulk only, low concurrency, off-peak, proxied; never the default path; feature-flag kill switch | Layer 1 |
| LLM hallucination in pitches | **High** | pstack: claims require proof; copy grounded only in VerifiedClaims; ECC outbox gate rejects unproven numbers; human approval until trust curve promotes | Layer 2 |
| Tenant data mixups and privacy | **High** | Tenant partition on every query and queue message; data/ and logs/ untracked from git (audit D-finding); PII minimization in exports | Layer 3 + Layer 5 |
| Scraper rot after site redesigns | Medium | Scrapling adaptive selectors re-find by similarity; Firecrawl change monitors re-audit watchlist sites; fixture smoke tests catch drift in CI | Layer 1 |
| Vendor key sprawl | Medium | treg-pattern gateway: keys injected server-side, never in agent code or .env; price metadata on every tool call | Layer 4 |
| Lost or duplicated work on restart | Medium | Idempotent job queue (Redis + arq); outbox pattern with keyed sends; reconciliation sweep for missed webhooks | Layer 5 |
| Over-automation damaging reputation | Medium | Trust curve: every automation stage purchased with measured evidence; one-flag rollback per stage; QABot conversation sampling | Layer 5 |

Table 9.1 - Risk register: every existential risk paired with its designed countermeasure and owner.

The WhatsApp row deserves one more paragraph of plain talk, because it is the risk most likely to be underestimated. WhatsApp Business messaging is a permissioned channel: templates are reviewed, quality ratings are tracked per number, and a number that sends unapproved copy or ignores opt-outs gets restricted, then banned, and a banned sender is a dead business asset in a market where the number is printed on the shop board. The architecture therefore treats the send path as the most governed edge in the system: nothing sends without a bundle id, nothing sends outside quiet hours, nothing sends twice because sends are keyed, and the nightly audit reviews the quality metrics alongside the leads. When the nightly report says "template quality dropped on tenant B", the correct response is to slow that tenant's ramp, not to argue with the platform. The machine scales only as fast as its reputation allows - that sentence is Layer 5 in six words.

# 10. The Master Library - Every Deliverable and When to Open Which

Four documents now cover the whole project, each with one job, written to interlock rather than overlap. This blueprint is **the map**: what the finished machine is and how the parts connect. The Master Action Plan is **the build order**: phases 0 through 5 with commands, acceptance checks, and the master checklist you tick every session. The audit report is **the why**: every defect, score, and evidence trail behind the repair decisions. The Agent Engineering Playbook is **the theory**: the Lauren Tan masterclass, the bot marketplace patterns, and the trust methodology the whole loop implements. Read in the right order for the right job, they replace the need to re-derive any decision from scratch.

| Document | Its job | Open it when |
|---|---|---|
| **IC_Unified_System_Architecture.pdf** (this document) | The map: inventory, five layers, contracts, sprints 5-8, verticals, guardrails | Planning integration work; onboarding; any "which component does X" question |
| IC_Master_Action_Plan.pdf | The build order: phases 0-5, commands, acceptance checks, Master Checklist | Every work session - start here, always |
| IC_Intelligence_Center_Audit_and_Upgrade_Report.pdf | The why: defects D1-D8, scorecard, UI blueprint, evidence | You need the proof behind a step or a defect |
| Agent_Engineering_Playbook_Research_and_Build_Blueprint.pdf | The theory: Lauren Tan loop, bot marketplace, trust methodology | Designing new bots or skills; explaining the method to others |

Table 10.1 - The master library: four documents, four jobs, one project.

| Situation | Reading order | Time |
|---|---|---|
| New teammate day one | This Ch. 1-4, then Master Plan Ch. 1-2 | 30 minutes |
| Starting a repair session | Master Plan phase chapter, then its checklist items | 5 minutes |
| Starting an integration sprint | This Ch. 5-7, then Master Plan Ch. 11 checklist | 20 minutes |
| Designing a new bot or agent | Playbook full read, then this Ch. 3 Layer 5 | 1 hour |
| Explaining the system to a client | This Ch. 1 and Ch. 6 only | 10 minutes |

Table 10.2 - Reading orders by situation: each job needs only two stops.

The closing perspective is the one this document opened with, now earned: **you own one machine, not seven repositories.** The hands find businesses; the brain proves what they need; the memory never forgets a touch; the mouth and ears hold the conversation; the conscience gates every merge and every send. The repair phases make the core trustworthy, the integration sprints make it whole, and the trust curve makes it progressively autonomous exactly as fast as its evidence allows. The method that runs it is one line you already know: verification first, then trust, then hard CI rules, then auto-merge. Follow the map, tick the checklist, and let the machine earn its stages - that is the whole discipline.